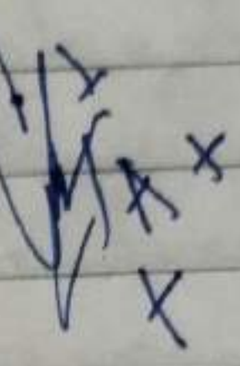
$x = [1,2,3]$

Modelo de programación funcional

Es un paradigma de programación declarativa basado en el uso de funciones matemáticas puras para transformar datos, en lugar de ejecutar una secuencia de instrucciones paso a paso.

Pilares fundamentales



- Funciones puras: Son bloques de código que siempre devuelven el mismo argumentos, y no generan efectos secundarios.
- Inmutabilidad: Los datos no se pueden modificar una vez creados; si necesitas cambiar un valor, se genera una estructura de datos nueva con la modificación.
- Funciones de orden superior: Permiten pasar funciones como argumentos o otras funciones, o devolverlas como resultados (ejemplos comunes son map, filter y reduce).
- Composiciones de funciones: Permite unir varias funciones simples para construir operaciones más complejas y legibles.

Entendemos por programación funcional un lenguaje de programación declarativo donde el programador especifica lo que quiere hacer, en lugar de lidiar con el estado de los objetos y sea más legible.

Es decir, las funciones estarían en un primer lugar y nos centraremos en expresiones.

Scribe®

Cálculo Lambda - Alonzo Church

El cálculo lambda se origina en una época donde los matemáticos buscaban formalizar la noción de función y computación.
Alonzo Church, un lógico y matemático estadounidense, introdujo este sistema en su búsqueda por establecer una teoría de computación que fuera más revolucionaria que las alternativas disponibles en su tiempo

Fundamentos del cálculo lambda.

Para entender el cálculo lambda, es fundamental familiarizarse con sus elementos básicos: los términos del cálculo. En este contexto, los términos se clasifican en variables aplicaciones y abstracciones

Una variable representa un identificador, una abstracción permite definir funciones y una aplicación expresa la invocación de funciones. Este equilibrio entre variables, aplicaciones y abstracciones crea un sistema altamente expresivo que se ocupa de la manipulación de funciones sin la necesidad de estados o estructuras complejas.

La notación del cálculo lambda se representa en la forma $\lambda x.M$, donde x es una variable y M es un término. Este formalismo permite expresar funciones de manera simple y concisa, pero sus características van más allá de la simple notación.

Con el código lambda (cálculo lambda) las funciones pueden ser creadas, aplicadas, y también pasadas como argumentos, lo que introduce la capacidad de tratar las funciones como datos.
Esto es crítico en varios paradigmas de programación funcional, donde se prioriza la pureza de las funciones sobre los efectos secundarios.

Una variante notable es el cálculo lambda de Church, que enfatiza la importancia de las funciones sin la necesidad de construir una estructura lógicamente abstracta alrededor de ellas. Además, el cálculo lambda puede dividirse en dos categorías: el cálculo lambda extendido. La versión extendida incluye conceptos como manejo de excepciones, lo que permite que los programas sean más apropiados para situaciones del mundo real donde los errores pueden ocurrir durante la ejecución.

Aplicaciones del cálculo lambda en la informática:

La influencia del cálculo lambda se extiende a muchos campos dentro de la informática y la programación. Uno de los usos más reconocidos está en los lenguajes de programación funcional, tales como Haskell, Lisp y Scala.
Estos lenguajes toman los fundamentos del cálculo lambda y los implementan en su propia sintaxis y semántica.

Erlang: lenguaje funcional y concurrente. Usa procesos hiperligeros independientes del sistema operativo que se comunican enviando mensajes. Sigue la filosofía "déjalo fallar" (si un proceso peta, se reinicia solo sin tumbar el sistema).

1. El modelo de actores (Concurrencia Real)

Erlang no utiliza hilos (threads) del sistema operativo para ejecutar tareas simultáneas, si no procesos ultraligeros creados dentro de su propia máquina virtual (BEAM)

2. Tolerancia a fallo: "Let it CRASH" (Déjalo fallar)

En la mayoría de los lenguajes, un programador pasa mucho tiempo escribiendo código para atrapar errores (try/catch) y evitar que la aplicación se cierre.
Erlang piensa al revés: asume que las cosas van a fallar.

- Supervisores: Los procesos comunes realizan el trabajo sucio. Por encima de ellos hay procesos llamados Supervisores, cuya única tarea es vigilar a los trabajadores

- Reinicio automático: Si un proceso encuentra un error inesperado, simplemente muere (crashea).
El supervisor detecta la muerte del proceso al instante y lo reinicia desde un estado limpio y seguro.
Todo esto ocurre en milisegundos sin que el usuario final note una interrupción.

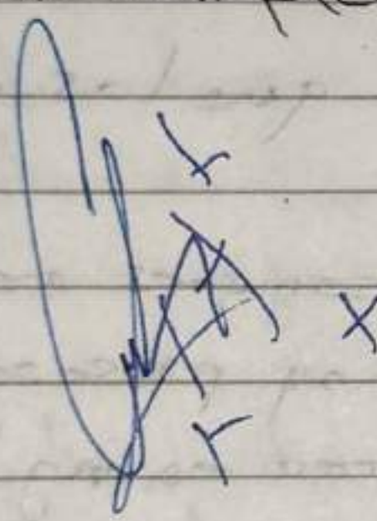
3. Programación funcional e inmutabilidad

Erlang no tiene variables en el sentido tradicional; tiene asignaciones únicas. Una vez que defines el valor de algo, nunca cambia (es inmutabilidad).

Elixir: Hereda toda la potencia, velocidad y estabilidad, de Erlang, pero con herramientas modernas para desarrolladores y código extensible.

Elixir es un lenguaje de programación moderno, dinámico, y funcional creado en 2014 por el desarrollo brasileño José Valim.
Su propósito principal fue resolver un gran dilema: cómo aprovechar la increíble potencia, velocidad y estabilidad de Erlang, pero utilizando una sintaxis moderna, amigable y productiva para los desarrolladores.

Para lograrlo, Elixir se diseñó para correr sobre BEAM, la misma máquina virtual de Erlang. Esto significa que Elixir es Erlang por dentro, pero con un diseño exterior completamente renovado.



Funciones de orden superior.

Son aquellas que cumplen al menos que tenga una de estas condiciones: reciben otra función como argumento o devuelven una función como resultados.

Características principales

- Tratamiento de funciones como ciudadanos de primera clase: Las funciones se pueden asignar a variables, guardar en estructura de datos, pasar como argumentos y retornar de la misma manera que cualquier otro dato.

Reutilización y modularidad del código: Permite abstraer patrones repetitivos. En lugar de escribir bucles o condicionales complejos una y otra vez, se encapsula la lógica genérica y se inyecta el comportamiento específico mediante funciones auxiliares.

- Composición de funciones: Facilita la unión de varias funciones más compleja y legible.

- Abstracción de operaciones comunes: En programación funcional se utilizan de forma masiva para procesar colecciones de datos mediante métodos estándar como map(), filter() y reduce().

2.3 Intervalos

Un intervalo (en inglés range) es una notación abreviada para construir listas de valores consecutivos sin escribirlos uno por uno.
Su forma básica es [inicio..fin], y funciona con cualquier tipo que pertenezca a la clase Enum: enteros, caracteres, booleanos, y también los tipos propios que declares con deriving Enum.
Por debajo, Haskell traduce la notación a funciones de esa clase [a..b] es enumFromTo a b, [a,b..c] es enumFromThenTo a,b,c, [a,b..] es enumFrom a y [a,b...] es enumFromThen a b.

Cuando se escriben los dos primeros elementos, Haskell calcula el paso a partir de la diferencia entre ambos, lo que permite saltos mayores a 1 y también secuencias descendentes. Para contar hacia atrás siempre hay que indicar el paso, porque [10..1] devuelve una lista vacía, ya que el paso por defecto es +1. Si se omite el límite superior se obtiene una lista infinita, que se puede usar sin problema gracias a la evaluación perezosa.

```
main :: IO()
main = do
    print [1..5]        -- [1,2,3,4,5]
    print [2,4..10]     -- [2,4,6,8,10]
    print [10,8..2]     -- [10,8,6,4,2]
    print ['a'..'e']    -- "abcde"
    print (take 3 [1..])  -- [1,2,3]
```

2.4 Operadores

En haskell un operador es una función cuyo nombre está formado por símbolos y que se escribe entre sus dos argumentos (forma infija).
No hay ninguna diferencia esencial entre el operador y una función normal: 3+4 es exactamente lo mismo que (+) 3 4, y cualquier función de dos argumentos puede usarse como operador encerrado su nombre entre acentos graves, como en 10 `div` 3.

Los operadores de uso común se agrupan por su función. Los aritméticos son +, -, * y / (división decimal), junto con div y mod para división y otros enteros, y tres formas de potencia: ^ para exponentes enteros no negativos, ^^ para exponentes enteros que pueden ser negativos y ** para exponentes decimales. Los de comparación son == y /= (distinto no se escribe !=), <, <=, > y >=. Los lógicos son && (y), || (o) y la función not. Para listas se usan: ++ (agrego un elemento al inicio), ++ (concatena dos listas) y !! (obtiene el elemento en una posición contando desde cero)

2.7. Evaluación Perezosa

Haskell usa evaluación perezosa, también llamada call-by-need: una expresión no se calcula en el momento en que se escribe, si no únicamente cuando su valor hace falta y una vez calculado el resultado se guarda para no repetir el trabajo.
Internamente, cada calculo pendiente se representa con un thunk, una especie de promesa que se resuelve la primera vez que alguien pide el valor.

Las primeras consecuencias es que se pueden estructuras infinitas y consumir solo la parte necesaria.
Una definición como naturales = [1..] no se queda calculando para siempre porque nadie pide la lista completa, y take 5 naturales solo produce 5 elementos.
La segunda consecuencia es que los argumentos no se usan nunca se evalúan, por lo que no pueden provocar errores ni gastar tiempo.
La tercera es que el cortocircuito de && y || no necesitan un tratamiento especial en el lenguaje False && undefined devuelve False simplemente porque el segundo argumento no hace falta.
La cuarta es el comportamiento de resultados: si pa valor se usa varias veces dentro de una definición, se calcula una sola vez.

	Notación Prefija	Notación Infija	Notación Postfija
Definición	La función se posiciona antes de todos sus argumentos, es el estándar absoluto en Haskell.	La función se posiciona entre sus dos argumentos, está diseñada específicamente para operaciones binarias.	La función se posiciona después de su argumento. Haskell carece de soporte sintáctico nativo para la notación postfija.
Comportamiento nativo	Actúa por defecto en funciones alfanuméricas. (letras)	Actúa por defecto en operadores simbólicos (+, *, &&).	No existe de forma nativa en la sintaxis de Haskell.
Sintaxis estándar	función arg1 arg2 ... argN	Arg1 función Arg2	—
Aridad permitida	Cualquiera (0, 1, 2, ..., n argumentos).	Estrictamente dos argumentos (operaciones binarias).	Normalmente un argumento a la vez.
Mecanismo de conversión	Envuelve símbolos en paréntesis: (+) 5 3.	Envuelve letras entre comillas invertidas (back-ticks): 5 `mod` 3	Requiere importar el operador (&) de Data.Function.
Mayor ventaja	Permite aplicar currificación y aplicar funciones parcialmente.	Emula la sintaxis matemática tradicional, mejorando la lectura	Elimina paréntesis anidados al crear canales de datos.
Ejemplo.	div 10 2 filter esPar [1..10]	10 `div` 2 2 < 5 + 3	10 & (1x => x/2) [1..10] & filter esPar.